프루닝 사전 지식

Antara NPU 용 YOLOv3 압축 모델을 이해하기 위한 최소 지식 · 2026-08-02

> **한 줄 요약**
> % 는 **얼마나 줄였나**(압축률), **step(계단 폭)** 은 **몇 개씩 끊어서 줄였나**(단위)입니다. NPU 는 채널을 **64 개 단위**로 처리하므로, 64 의 배수로 끊어 줄이지 않으면 **줄여도 빨라지지 않습니다**.

1. 프루닝이란

학습된 신경망에서 **덜 중요한 부분을 잘라내** 모델을 작고 빠르게 만드는 기법입니다. 두 갈래가 있습니다.

방식	내용	NPU 적합성
비정형 unstructured	개별 가중치를 0 으로. 정확도 손실이 적지만 **구조가 그대로**라 0 을 건너뛰려면 전용 하드웨어가 필요	✗ 실제로 안 빨라짐
정형 structured	**채널·필터를 통째로 제거.** 텐서가 실제로 작아져 **어떤 하드웨어에서도 바로 빨라짐**	✓ 이 워크스페이스가 쓰는 방식

YOLOv3 압축 모델은 전부 **정형(채널) 프루닝**입니다. Conv 레이어 **개수는 75 개로 동일**하고 **채널 수만** 줄어듭니다. 그래서 구조·입출력이 원본과 같아 그대로 갈아끼울 수 있습니다.

2. % — 얼마나 줄였나

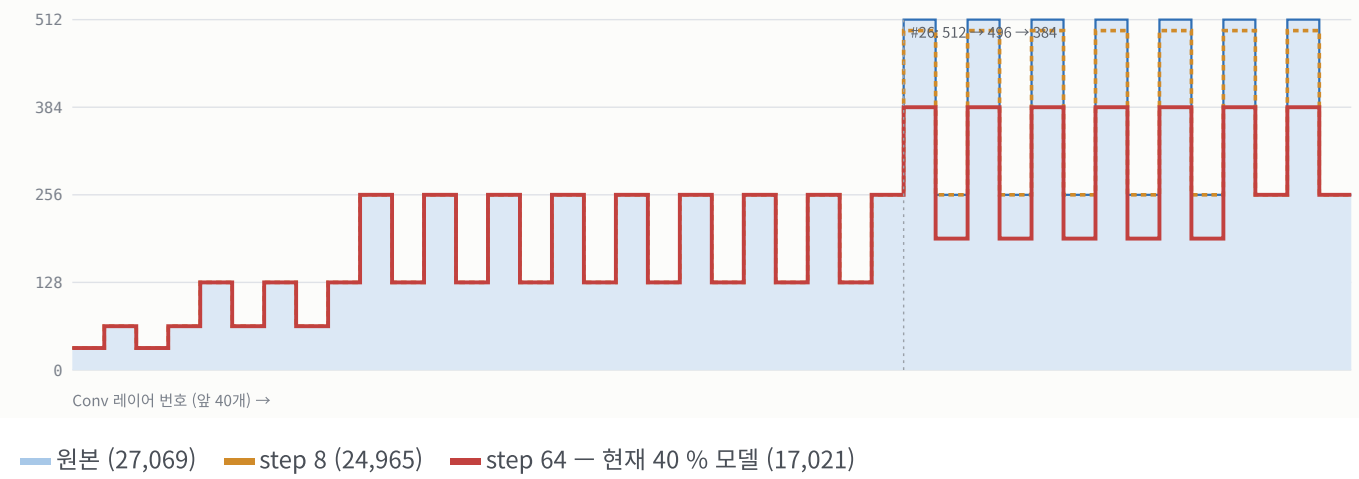
압축률입니다. 다만 **무엇을 기준으로 한 %인지** 주의해야 합니다.

모델	총 채널	채널 감소	파일 크기	크기 감소
원본	27,069	—	247.7 MB	—
40 % 프루닝	17,021	37.1 %	81.5 MB	67.1 %
50 % 프루닝	14,781	45.4 %	59.0 MB	76.2 %

> **이름의 % 와 실제 채널 감소가 다릅니다.** “40 %” 모델의 채널은 37.1 % 줄었고 파일은 67 % 줄었습니다. 파라미터 수 기준 표기로 보입니다 — 채널을 c 배로 줄이면 Conv 가중치는 대략 c^2 배로 줄기 때문입니다. **% 만 보고 속도나 메모리를 예측하면 어긋납니다.**

3. step(계단 폭) — 몇 개씩 끊어서 줄였나

채널을 **몇 개 배수 단위로** 깎느냐입니다. 파일명 `pruned40%_step64_retrain` 의 `step64` 가 이것입니다. 레이어별 채널 수를 그리면 **계단 모양**이 되고, 그 계단 한 칸의 높이가 step 입니다.



같은 레이어에서 어떻게 달라지는지 보면 분명합니다.

```
레이어 #26     원본 512  →  step 8 : 496  (16 개만 깎음)
                          →  step 64: 384  (128 개 통째로)
```

왜 64 인가

Antara NPU 의 하드웨어 제원이 그렇습니다.

항목	값	의미
TX array (행렬 연산기)	64 × 32	한 번에 64 채널을 처리
SRAM word 크기	64 B	메모리 접근 단위
SRAM 가중치 대역폭	64	가중치 읽는 폭

> **496 채널로 깎아도 하드웨어는 512 칸을 씁니다.** 연산량은 그대로인데 정확도만 잃습니다. 실제로 `step 8` 모델은 채널을 7.8 % 줄이는 데 그쳤으면서 64 배수 정렬이 깨진 Conv 가 **5 개에서 31 개로** 늘었습니다. 이후 결과물이 전부 `step64` 인 이유입니다.

모델	총 채널	원본 대비	64 배수 아닌 Conv
원본	27,069	100.0 %	5 / 75
step 8	24,965	92.2 %	**31 / 75** ← 깨짐
step 64 (40 %)	17,021	62.9 %	5 / 75 ← 유지

4. retrain — 잘라낸 뒤 다시 학습

채널을 잘라내면 정확도가 떨어지므로 **전체를 다시 학습**해 회복시킵니다. 파일명의 `retrain` 이 그 표시입니다.

> 확인해 본 결과 — 프루닝 모델의 첫 Conv 가중치는 원본과 **상관계수 1.0000** 인데 값은 조금씩 다릅니다. 처음부터 다시 배운 게 아니라 **원본에서 출발해 미세 조정**되었다는 뜻입니다. 크기가 그대로인 27 개 Conv 도 전부 값이 바뀌어 있어, **네트워크 전체가 재학습**되었음을 알 수 있습니다.

5. 실제로 얼마나 손해인가

COCO val2017 500 장, float32 기준 실측입니다. 세 모델에 **같은 이미지**를 넣어 비교했습니다.

모델	AP	AP50	작은 객체 AP	속도	크기
원본	**40.72**	70.10	30.64	1.00×	247.7 MB
40 % 프루닝	33.75	57.26	20.49	**1.71×**	81.5 MB
50 % 프루닝	27.93	49.69	17.34	**1.91×**	59.0 MB

- **작은 객체가 먼저 무너집니다.** 작은 객체 AP 는 33 % / 43 % 떨어져 큰 객체(16 % / 34 %)의 두 배입니다. 채널을 줄이면 고해상도 헤드의 표현력이 먼저 희생됩니다.
- **50 % 는 수지가 맞지 않습니다.** 40 % → 50 % 로 갈 때 AP 는 5.8 점 더 잃는데 속도는 12 % 늘 뿐입니다. **40 % 가 균형점**으로 보입니다.
- 절대 AP 는 표본 500 장 기준이라 논문 수치(33.7)와 직접 비교할 수 없습니다. **모델 간 차이**를 보는 용도입니다.

6. 정리

용어	뜻	현재 모델
정형 프루닝	채널·필터를 통째로 제거. 레이어 수는 그대로, 채널만 감소	Conv 75 개 유지
%	압축률. **파라미터 수 기준**이라 채널 감소율과 다름	40 % / 50 % 두 가지
step (계단 폭)	채널을 깎는 배수 단위. **하드웨어 정렬**에 맞춰야 실제로 빨라짐	**둘 다 step 64**
retrain	잘라낸 뒤 전체 재학습해 정확도 회복	적용됨

> **용어 주의** — 이 문서의 “step / 계단 폭” 은 파일명 `step64` 에서 온 표현입니다. 워크스페이스의 스크립트·컴파일러·드라이버 어디에도 `staircase` 라는 용어는 없습니다. 같은 개념을 다른 이름으로 부르고 계신 것이라면 이 문서의 3 절이 그에 해당합니다.

근거 문서 — `workspace_docs/docs/pruning-map-comparison.md` (mAP 실측), `workspace_docs/model/README.md` (모델 출처·채널 통계), `workspace_docs/docs/antara-npu-internals.png` (NPU 제원)